

System Design Document

Autonomous Delivery Cart

Contents

- 1. Introduction2
 - 1.1 Purpose.....2
 - 1.2 Scope.....2
- 2. System Overview2
- 3. Architectural Design3
 - 3.1 High-Level Architecture Diagram3
 - 3.2 Component Description3
- 4. Database Design4
- 5. Data Flow Design.....4
- 6. Interface Design4
- 7. Security Design4
- 8. Scalability & Performance5
- 9. Future Enhancements5

1. Introduction

1.1 Purpose

This document describes the design and architecture of the Autonomous Delivery Cart System. It serves as a blueprint for implementing the system according to the Software Requirements Specification (SRS). It explains system components, data flow, AWS cloud infrastructure, interfaces, and database design.

The intended audience includes the project supervisor (Dr. Diaa), the development team, and any future contributors.

1.2 Scope

The system consists of:

- A **mobile application** for customers to register, place, and track delivery orders.
- A **web dashboard** for managers to monitor carts, vendors, and analytics.
- A **vendor portal** for markets and pharmacies to manage their inventory and dispatch orders.
- A **cloud backend** hosted on AWS that manages authentication, database, API logic, IoT communication, and analytics.
- A **smart cart** equipped with sensors (LiDAR, ZED, Kinect) that connects to the cloud and communicates in real time.

2. System Overview

The system follows a **modular microservice-oriented architecture** hosted on AWS.

It includes the following major subsystems:

Subsystem	Description
Mobile Application	Frontend for customers to browse vendors, place orders, and track carts.
Vendor Web Portal	Allows vendors to manage inventory and send delivery commands to carts.
Manager Dashboard	Provides administrative controls, analytics, and operational insights.
AWS Cloud Backend	Manages APIs, authentication, database, and IoT communication.

Smart Cart	Hardware device that communicates via AWS IoT Core and executes delivery routes.
-------------------	--

3. Architectural Design

3.1 High-Level Architecture Diagram

Components:

- **Frontend Layer:** Mobile (Flutter), Web Dashboard
- **Backend Layer:** API Gateway, Lambda functions, DynamoDB, S3, Cognito
- **IoT Layer:** AWS IoT Core, MQTT Topics, Jetson hardware
- **Data Analytics Layer:** AWS QuickSight, CloudWatch Logs, S3 Data Lake
- **Infrastructure Management:** AWS CloudFormation or Terraform for IaC

3.2 Component Description

Component	Technology	Purpose
Amazon Cognito	AWS Service	Manages user authentication for customers, vendors, and managers.
Amazon API Gateway	AWS Service	Acts as entry point for mobile and web requests to backend functions.
AWS Lambda	Python/Node.js	Executes backend business logic (orders, cart updates, analytics).
AWS IoT Core	AWS Service	Manages communication between cart hardware and cloud via MQTT.
Amazon DynamoDB	NoSQL	Stores user profiles, orders, and IoT telemetry.
Amazon S3	Object Storage	Stores images, logs, and archived sensor data.
Amazon CloudWatch	Monitoring	Tracks system logs, metrics, and IoT device health.
Amazon QuickSight	BI Tool	Provides visual analytics and dashboards.
AWS Amplify	Hosting	Deploys and manages frontend (dashboard, web apps).

4. Database Design

The database uses a **hybrid approach**:

- **Relational data** (users, orders, vendors) → DynamoDB structured items
- **Sensor/IoT data** → S3 bucket (for logs and analytics)

Key Entities:

- Users
- Vendors
- Orders
- Carts
- CartTelemetry (battery, GPS, status)
- Inventory

5. Data Flow Design

- Customer places order → API Gateway → Lambda → DynamoDB
- Vendor confirms → Lambda → IoT Core publishes “start_delivery”
- Cart moves → sends telemetry every 30s → IoT Core → DynamoDB
- Upon arrival → user receives SMS code → enters it → Lambda verifies → IoT unlocks
- Cart returns → data logged → CloudWatch + QuickSight analytics update

6. Interface Design

- **Mobile App UI:** Simple navigation for order placement, cart tracking, and notifications.
- **Vendor Portal UI:** Inventory management, order acceptance, and delivery dispatch.
- **Dashboard UI:** Overview of active carts, completed deliveries, and analytics.

7. Security Design

- All API requests use **HTTPS (TLS 1.2)**.

- **Cognito** manages role-based access (Customer, Vendor, Manager).
- **IoT policies** restrict each cart to its own MQTT topic.
- **CloudWatch logs** audited for suspicious activity.

8. Scalability & Performance

- Uses **serverless architecture** (Lambda + API Gateway) for auto-scaling.
- DynamoDB and IoT Core scale automatically to handle increased users or carts.
- S3 lifecycle policies to archive old sensor data cost-effectively.

9. Future Enhancements

- Add online payment gateway (Stripe / AWS Payments).
- AI-based route optimization.
- Computer vision-based obstacle detection and facial recognition for security.